

# Relatório de Testes Unitários

## Informações do Projeto

**Nome do Projeto:** projeto-testes-jest  
**Versão:** 1.0.0  
**Framework de Testes:** Jest v29.0.0  
**Data:** 11/05/2026

## Estrutura do Projeto

```
projeto-testes-jest/  
├── src/  
│   └── services/  
│       ├── auth.js           ← Módulo de autenticação  
│       ├── shipping.js       ← Módulo de cálculo de frete  
│       └── cart.js           ← Módulo de carrinho de compras  
├── tests/  
│   ├── auth.test.js  
│   ├── shipping.test.js  
│   └── cart.test.js  
├── package.json  
└── README.md
```

## Resultados dos Testes

**Total de Testes:** 9  
**Testes Passados:** 9  
**Testes Falhados:** 0  
**Sucesso:** 100%

```
Test Suites: 3 passed, 3 total  
Tests:      9 passed, 9 total  
Time:       ~2.3s
```

## Módulo 1: Autenticação (auth.js)

### Funções Implementadas

#### validateEmail(email)

- Valida o formato de e-mail
- Retorna `true` se o e-mail for válido
- Retorna `false` para e-mails vazios, sem @, ou sem domínio

```
function validateEmail(email) {  
  if (!email || email === "") return false;  
  if (!email.includes("@")) return false;  
  
  const parts = email.split("@");  
  if (parts.length !== 2 || parts[1] === "" || parts[0] === "") {  
    return false;  
  }  
  
  return true;  
}
```

#### login(email, password)

- Realiza autenticação com usuários mockados
- Retorna `{ success: true, token: "token-simulado-xxxx" }` para credenciais válidas
- Retorna `{ success: false, message: "Credenciais inválidas" }` para senha incorreta
- Retorna `{ success: false, message: "E-mail inválido" }` para e-mail inválido

```
function login(email, password) {
  if (!validateEmail(email)) {
    return { success: false, message: "E-mail inválido" };
  }

  const user = users.find(u => u.email === email);

  if (!user || user.password !== password) {
    return { success: false, message: "Credenciais inválidas" };
  }

  return { success: true, token: `token-simulado-${Date.now()}` };
}
```

## Testes Implementados (3 testes)

### Teste 1: Sucesso na autenticação

```
test('login() retorna success: true e token quando credenciais são válidas', () => {
  const email = "admin@email.com";
  const password = "admin123";

  const result = login(email, password);

  expect(result.success).toBe(true);
  expect(result).toHaveProperty('token');
  expect(result.token).toMatch(/^token-simulado-/);
});
```

Status: ☒ PASSOU

### Teste 2: Falha com senha incorreta

```
test('login() retorna success: false e mensagem "Credenciais inválidas" com senha errada', () => {
  const email = "admin@email.com";
  const password = "senha_errada";

  const result = login(email, password);

  expect(result.success).toBe(false);
  expect(result.message).toBe("Credenciais inválidas");
});
```

Status: ☒ PASSOU

### Teste 3: Validação de e-mail vazio

```
test('validateEmail("") retorna false para e-mail vazio', () => {
  const email = "";

  const result = validateEmail(email);

  expect(result).toBe(false);
});
```

Status: ☒ PASSOU

---

## Módulo 2: Cálculo de Frete (shipping.js)

### Funções Implementadas

#### calculateShipping(cep, weight)

- Calcula frete baseado nos primeiros 2 dígitos do CEP
- **CEP 01-19 (SP):** R\$ 10,00 base + R\$ 1,50 por kg
- **CEP 20-29 (RJ/ES):** R\$ 15,00 base + R\$ 2,00 por kg
- **Outros CEPs:** R\$ 20,00 base + R\$ 2,50 por kg
- Lança `Error("CEP inválido")` para CEP inválido

```
function calculateShipping(cep, weight) {
  const cepRegex = /^\\d{8}$/;
  if (!cepRegex.test(cep)) {
    throw new Error("CEP inválido");
  }

  const firstTwoDigits = parseInt(cep.substring(0, 2));
  let baseCost, costPerKg, region;

  if (firstTwoDigits >= 1 && firstTwoDigits <= 19) {
    baseCost = 10.00;
    costPerKg = 1.50;
    region = "SP";
  } else if (firstTwoDigits >= 20 && firstTwoDigits <= 29) {
    baseCost = 15.00;
    costPerKg = 2.00;
    region = "RJ/ES";
  } else {
    baseCost = 20.00;
    costPerKg = 2.50;
    region = "Outros";
  }

  const totalCost = baseCost + (weight * costPerKg);

  return {
    region,
    cost: parseFloat(totalCost.toFixed(2))
  };
}
```

#### isFreeShipping(orderTotal)

- Retorna true se o total do pedido for >= R\$ 200,00
- Retorna false caso contrário

```
function isFreeShipping(orderTotal) {
  return orderTotal >= 200.00;
}
```

### Testes Implementados (3 testes)

#### Teste 1: Frete para SP

```
test('calculateShipping("01310100", 2) retorna custo correto para região SP', () => {
  const cep = "01310100";
  const weight = 2;

  const result = calculateShipping(cep, weight);

  expect(result.region).toBe("SP");
  expect(result.cost).toBe(13.00);
});
```

Status: ☒ PASSOU

#### Teste 2: Frete grátis acima de R\$200

```
test('isFreeShipping(250) retorna true', () => {
  const orderTotal = 250;

  const result = isFreeShipping(orderTotal);

  expect(result).toBe(true);
});
```

Status: ☒ PASSOU

#### Teste 3: CEP inválido lança erro

```
test('calculateShipping("ABC", 1) lança Error com mensagem "CEP inválido"', () => {  
  const cep = "ABC";  
  const weight = 1;  
  
  expect(() => calculateShipping(cep, weight)).toThrow("CEP inválido");  
});
```

Status: ☒ PASSOU

## Módulo 3: Carrinho de Compras (cart.js)

### Funções Implementadas

#### addItem(cart, item)

- Adiciona item ao carrinho
- Se o item já existir (mesmo id), incrementa a quantidade
- Caso contrário, adiciona o item ao array

```
function addItem(cart, item) {  
  const existingItemIndex = cart.findIndex(cartItem => cartItem.id === item.id);  
  
  if (existingItemIndex !== -1) {  
    cart[existingItemIndex].quantity += item.quantity;  
  } else {  
    cart.push({ ...item });  
  }  
  
  return cart;  
}
```

#### removeItem(cart, itemId)

- Remove o item com o id correspondente do carrinho
- Retorna o carrinho atualizado sem o item
- Se o id não existir, retorna o carrinho sem modificações

```
function removeItem(cart, itemId) {  
  return cart.filter(item => item.id !== itemId);  
}
```

#### calculateTotal(cart)

- Soma price \* quantity de todos os itens
- Retorna o valor total como número (float com 2 casas decimais)
- Retorna 0 para carrinho vazio

```
function calculateTotal(cart) {  
  if (!cart || cart.length === 0) {  
    return 0;  
  }  
  
  const total = cart.reduce((sum, item) => {  
    return sum + (item.price * item.quantity);  
  }, 0);  
  
  return parseFloat(total.toFixed(2));  
}
```

### Testes Implementados (3 testes)

#### Teste 1: Adicionar item novo

```
test('addItem() retorna array com 1 item ao adicionar item novo', () => {
  const cart = [];
  const item = { id: 1, name: "Produto A", price: 50, quantity: 1 };

  const result = addItem(cart, item);

  expect(result).toHaveLength(1);
  expect(result[0]).toEqual(item);
});
```

Status: ☒ PASSOU

#### Teste 2: Remover item existente

```
test('removeItem() remove item por ID do carrinho', () => {
  const cart = [
    { id: 1, name: "Produto A", price: 50, quantity: 1 },
    { id: 2, name: "Produto B", price: 30, quantity: 2 }
  ];

  const result = removeItem(cart, 1);

  expect(result).toHaveLength(1);
  expect(result[0].id).toBe(2);
});
```

Status: ☒ PASSOU

#### Teste 3: Total do carrinho

```
test('calculateTotal() retorna valor correto com múltiplos itens e quantidades diferentes', () => {
  const cart = [
    { id: 1, name: "Produto A", price: 50, quantity: 2 },
    { id: 2, name: "Produto B", price: 30, quantity: 1 }
  ];

  const result = calculateTotal(cart);

  expect(result).toBe(130.00);
});
```

Status: ☒ PASSOU

## Padrão Utilizado

Os testes foram escritos de forma direta e concisa, focando na funcionalidade testada sem comentários excessivos.

## O que está sendo testado?

- ✓ **Módulo de Autenticação (auth.js):** validação de e-mail, login com sucesso, login com falha
- ✓ **Módulo de Cálculo de Frete (shipping.js):** cálculo por região de CEP, frete grátis, CEP inválido
- ✓ **Módulo de Carrinho (cart.js):** adicionar item, remover item, calcular total

## O que NÃO está sendo testado?

- ✗ **Integração com Banco de Dados:** os testes usam dados mockados (hardcoded), sem conexão real
- ✗ **Interface Gráfica (UI):** apenas a lógica JavaScript é testada, sem renderização de componentes
- ✗ **Chamadas de API Externas:** respostas simuladas para evitar dependência de rede

## Como rodar os testes

### Instalar dependências:

```
npm install
```

**Rodar todos os testes:**

```
npm test
```

**Rodar com detalhes:**

```
npm run test:verbose
```

**Ver cobertura de código:**

```
npm run test:coverage
```

## Conclusão

Todos os 9 testes unitários foram implementados com sucesso, seguindo as melhores práticas de desenvolvimento e o padrão AAA. O projeto está pronto para uso educacional, demonstrando conceitos importantes de testes unitários com Jest.

**Status Final:** ☒ TODOS OS TESTES PASSARAM (9/9)